Conception et Exploitation d'une BDD

SQL Server • TP n°1 : Transaction et gestion des erreurs

Consignes Générales

Pour chaque exercice de ce TP, la démarche méthodologique suivante doit être appliquée :

- Créer les tables à partir du Modèle Logique de Données (MLD) fourni.
- Insérer les jeux de données nécessaires aux différents scénarios de test.
- Développer les procédures et blocs de traitement transactionnels demandés.
- Réaliser plusieurs tests rigoureux, incluant systématiquement des cas d'erreur.
- Vérifier l'état et l'intégrité de la base de données après chaque exécution.

POINTS CLÉS DE CONCEPTION

Il vous revient de déterminer précisément l'emplacement des instructions de contrôle transactionnel : où commence la transaction, où elle se valide (COMMIT), quelles opérations doivent être regroupées atomiquement, dans quelles situations annuler les modifications (ROLLBACK), et comment structurer la gestion des erreurs (TRY...CATCH).

Exercice 1 : Gestion d'un inventaire

CONTEXTE

Une entreprise commercialise du matériel informatique. Lorsqu'une commande est validée, plusieurs actions concomitantes doivent s'opérer : mise à jour de l'état de la commande, diminution des stocks disponibles, et enregistrement des mouvements de stock associés pour chaque produit commandé.

Règle métier critique : Une commande contenant plusieurs produits doit être traitée comme une seule unité logique (atomicité). L'entreprise refuse formellement qu'une commande soit partiellement validée.

Exercice 1 (Suite) : Analyse et Tests

ANALYSE DU BESOIN TRANSACTIONNEL

Pourquoi une erreur sur une seule ligne de commande ne doit pas laisser les modifications concernant les autres lignes en base ? Si une transaction globale n'encadrait pas l'ensemble des lignes, un stock insuffisant sur le second article laisserait le premier article décrémenté par erreur, corrompant l'inventaire. Le bloc transactionnel garantit un retour à l'état initial (rollback complet).

TESTS À RÉALISER

- Validation réussie d'une commande nominale.
- Commande comportant plusieurs produits distincts.
- Cas d'erreur : Quantité demandée supérieure au stock disponible.
- Cas d'erreur : Référence de commande inexistante.

Vérifications après tests : Contrôle rigoureux des tables `COMMANDE`, `PRODUIT` et `MOzUVEMENT_STOCK`.

Exercice 2 : Réservation de véhicules

CONTEXTE

Une société de location de véhicules souhaite automatiser la confirmation de ses réservations. Une réservation lie un client, un véhicule et une période précise. La confirmation valide la réservation, change le statut du véhicule en « réservé », enregistre le paiement associé et trace l'événement dans l'historique du véhicule.

> ### RÈGLES DE COHÉRENCE
>
> Le traitement doit valider l'existence de la réservation, s'assurer que le véhicule est bien disponible au moment de l'opération, et centraliser l'ensemble sous une transaction stricte pour éviter tout état intermédiaire (ex: paiement enregistré mais véhicule non bloqué).

Exercice 3 : Transfert d'un client

CONTEXTE

Une entreprise possède plusieurs agences commerciales. Chaque client y est rattaché. Lorsqu'un client change d'agence, l'entreprise souhaite conserver l'historique complet de ses affectations successives tout en mettant à jour sa fiche principale.

TRAVAIL DEMANDÉ & CONTRÔLES

Le traitement de transfert met en œuvre les étapes atomiques suivantes sous contrôle transactionnel :

- Vérification de l'existence du client et récupération de son agence actuelle.
- Vérification de l'existence de l'agence de destination.
- Contrôle de non-redondance (le client ne doit pas déjà être rattaché à l'agence cible).
- Clôture de l'affectation en cours dans l'historique (mise à jour de la date de fin).
- Création d'une nouvelle ligne d'affectation pour la nouvelle agence.
- Mise à jour de la référence de l'agence sur la fiche client.

VÉRIFICATION IMPORTANTE APRÈS ERREUR

En cas d'anomalie survenant au cours du transfert, il faut impérativement vérifier que :

- Le client n'a pas changé d'agence.
- Aucune affectation erronée n'a été insérée.
- L'historique n'a subi aucune modification partielle.

Question : Pourquoi l'ordre des opérations est-il crucial ? Clôturer l'ancienne affectation avant d'insérer la nouvelle avec la bonne date permet d'éviter des chevauchements de périodes d'affectation pour un même client.

Exercice 4 : Clôture d'une journée de billetterie

CONTEXTE

Chaque soir, une plateforme de billetterie réalise un traitement de clôture global pour les commandes de la journée. Le système met à jour l'état des commandes, vérifie l'adéquation des paiements reçus, et consigne un récapitulatif financier global dans une table dédiée.

STRATÉGIE DE TRAITEMENT

- **Détection d'anomalies :** Regroupement des commandes du jour pour vérifier que la somme des paiements valides correspond strictement au montant attendu de la commande. En cas d'écart, une exception est levée.
- **Mise à jour groupée :** Passage des commandes éligibles du statut 'EN ATTENTE' vers 'CLOTUREE'.
- **Consolidation :** Calcul du nombre total de commandes clôturées et du montant cumulé, puis insertion d'une ligne de synthèse dans le journal de clôture.

QUESTION DE CONCEPTION

Une erreur concernant une seule commande doit-elle nécessairement annuler toute la clôture ? D'un point de vue strict de comptabilité journalière, oui, pour garantir l'intégrité et la balance exacte des comptes. D'un point de vue opérationnel, certaines architectures choisissent d'isoler la commande anormale dans une table de quarantaine et de valider le reste, moyennant une validation métier explicite.

Exercice 5 : Analyse, correction et bilan

ANALYSE CRITIQUE DU CODE INITIAL

Un développeur affirmait qu'une transaction n'est pas nécessaire puisque SQL Server signale les erreurs.
Erreur majeure : Sans transaction explicite, si la modification de la commande réussit, que la mise à jour du stock réussit, mais que l'insertion du mouvement de stock échoue, la base se retrouve dans un état partiellement exécuté (stock faussé, absence de traçabilité).

BILAN DES CONNAISSANCES

Concept	Rôle et Définition
Transaction	Unité logique de travail regroupant plusieurs opérations devant toutes réussir ou toutes échouer ensemble.
COMMIT / ROLLBACK	`COMMIT` valide définitivement les modifications ; `ROLLBACK` annule l'ensemble des opérations de la transaction en cours.
TRY...CATCH	Mécanisme de capture et de gestion structurée des erreurs d'exécution pour intercepter les exceptions et déclencher le rollback.

Pourquoi une transaction doit-elle être courte ? Pour éviter de verrouiller les ressources trop longtemps, limiter les attentes concurrentes, réduire les risques de blocage (deadlocks) et préserver les performances globales du serveur.